

MEMO: Multimodal Evidence Memory Organization for Long-Horizon LLM Agents

Xian Gao^{1*}, Jinpeng Wang^{2*}, Jiacheng Ruan¹, Guangyu Cao³, Ting Liu^{1†}, and Yuzhuo Fu^{1†}

¹Shanghai Jiao Tong University, ²Tsinghua University,

³Institute of Automation of the Chinese Academy of Sciences

Abstract

Long-running LLM agents rely on external memory to store and reuse information beyond a single context window, yet there is a fundamental tension between the continuous accumulation of interaction trajectories and the limited context capacity. The key challenge in agent memory is therefore not only to retrieve relevant records, but also to select necessary evidence under a given budget and organize it in an appropriate modality. Existing memory readout methods mainly use textual or visual forms. Text preserves high fidelity, but its linear token representation makes contents with different importance compete for the limited context at nearly uniform unit cost. Visual readout renders text into document-like images, which can use two-dimensional layouts to expose structure and emphasize key information, but it may lose fine-grained details during rendering and compression. To address this issue, we propose **MEMO**, a Multimodal Evidence Memory Organization method for LLM agents. MEMO first uses a trained evidence extractor to select relevant memory blocks and form evidence units with source information and presentation requirements. A trained query-conditioned memory manager assigns each unit to a textual, visual, or dual-channel carrier and selects a layout that matches the evidence structure. A deterministic memory construction module then generates the textual package and visual pages. The memory manager is trained with feedback from an offline reader that measures the utility of the guided memory plan, so that retention and presentation decisions align with downstream usage. We evaluate MEMO on four benchmarks, HotpotQA, 2WikiMultiHopQA, LoCoMo, and ALFWorld, with multiple reader backends. The results show that MEMO presents memory more efficiently with fewer memory tokens, improves downstream task performance, and builds more effective working memory under constrained budgets. For example, under a fixed 128-token budget with Qwen3-VL-32B as the reader, MEMO achieves an F1 score of 73.91 on 2Wiki, outperforming text-only memory with 56.26 and visual-only memory with 35.89.

Introduction

Large language models (LLMs) are increasingly evolving into agents that continuously perceive their environments, invoke tools, and execute complex tasks. In long-horizon interactions, agents need external memory to store information beyond a single context window and to recall relevant records in later actions. However, as interaction trajectories accumulate, memory grows continuously. Feeding the full history

*These authors contributed equally.

†Corresponding author.

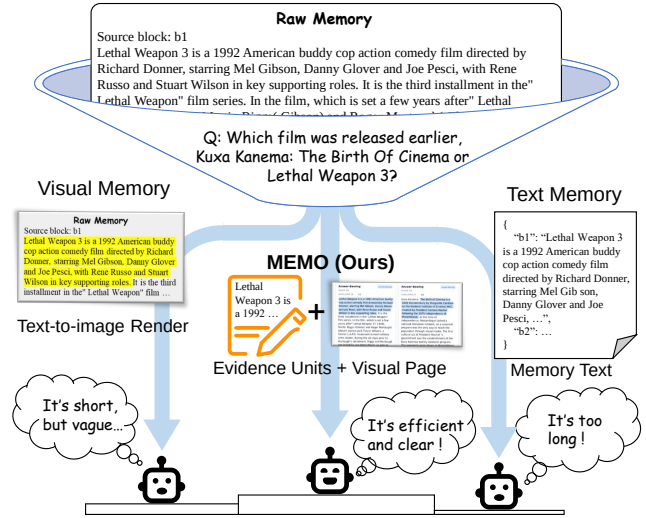


Figure 1: Motivation of MEMO.

directly into the model is not only costly, but also forces truly useful evidence to compete with a large amount of redundant or weakly relevant content for the limited context. The central challenge of long-term memory therefore goes beyond storage and retrieval. It also lies in memory readout: given the current query, candidate records, and a context budget, the memory system needs to select the evidence necessary for the task and organize it into a working memory with clear structure that the model can read efficiently.

As illustrated in Figure 1, existing methods implement memory readout mainly in textual or visual form. Textual memory systems build compact contexts through segmentation, compression, and retrieval. For example, SimpleMem and LightMem compress and retrieve historical records; MemRL, Agentic Memory, and Mem- α learn record utility or memory operations; EverMemOS and MemoryOS further manage memory hierarchies and life cycles. However, their final readout is still represented as a linear sequence of text tokens, which makes contents with different importance compete for context space at nearly uniform unit cost. Visual memory offers a complementary solution. MemOCR renders textual memory into visual pages with layout structure, while

AgentOCR encodes agent histories into OCR-style images through optical self-compression. Image rendering can highlight important evidence by emphasizing text spans and can represent information through page layouts, thereby reducing the number of tokens needed to express the same content. However, after text is rendered into images, fine-grained semantic information may be difficult to preserve accurately. These observations suggest that textual and visual representations are suited to different types of evidence. Restricting all memory to a single modality cannot simultaneously achieve information fidelity, structural expressiveness, and resource efficiency.

To address this issue, we propose MEMO, a multimodal evidence-based memory organization method. Its core idea is to jointly decide what to preserve and how to present it at the granularity of evidence units, rather than converting retrieved results into a single modality. Specifically, the trained evidence extractor in MEMO reads the query and candidate records, selects relevant source blocks, and predicts character-level spans that contain key facts. The selected spans and the original records together form traceable evidence units. Each unit preserves its source, specifies which details must be kept exactly, and indicates whether visual structures such as tables, timelines, lists, or cards can improve readability. This design protects key semantic information and provides compact operational objects for subsequent management. For each evidence unit, the trained memory manager jointly selects a text, image, dual-channel, or deletion action under a unified resource budget. The text representation preserves precise semantics, the visual representation presents structural relations through different layouts, and the dual-channel representation supplements visual structure while retaining textual anchors for semantic precision. The memory manager is trained with offline evaluations of candidate memory plans by a frozen reader model, which aligns evidence selection and presentation decisions with downstream task utility. Finally, the working memory building process constructs the working memory from the evidence units and their presentation actions, and verifies the total resource usage of textual and visual content.

We evaluate MEMO on HotpotQA (Yang et al. 2018), 2WikiMultiHopQA (Ho et al. 2020), LoCoMo (Maharana et al. 2024), and ALFWorld (Shridhar et al. 2020), covering multi-hop document question answering, long-term dialogue memory, and embodied interaction trajectory memory. We use multiple frozen VLMs as backend reader models. The experimental results show that MEMO achieves the best performance across all three reader models considered in the paper, both under a fixed 128-token budget and without a token budget. This indicates that evidence-level modality assignment and layout organization improve the use of limited context. The advantage is especially clear in multi-hop document question answering. With Qwen3-VL-32B as the reader model, MEMO achieves an F1 score of 73.91 on 2Wiki, while purely textual and purely visual memory achieve F1 scores of 56.26 and 35.89, respectively.

This paper makes three main contributions. First, we formulate agent memory readout as query-conditioned working memory construction under a shared resource budget, in-

tegrating evidence selection, presentation modality assignment, and visual organization into a unified optimization framework. Second, we propose MEMO, a multimodal memory organization method centered on traceable evidence units. By combining learned evidence extraction, memory planning, and deterministic working memory construction, MEMO exploits the complementarity between textual and visual structures, reducing the number of tokens required for memory while preserving semantic information. Third, we systematically evaluate MEMO across diverse memory sources and reader models, covering downstream task performance, resource usage, and adaptation across readers. The results validate the effectiveness of MEMO for long-horizon agent memory organization.

Related Work

Textual External Memory

Textual external memory is the most common memory interface for long-horizon LLM agents. Recent systems learn when to store, retrieve, revise, and compress records. MemRL estimates memory utility through runtime reinforcement learning over episodic memory (Zhang et al. 2026). Agentic Memory places short-term and long-term memory operations in the agent action space (Yu et al. 2026). SimpleMem and LightMem emphasize compact memory construction, topic-aware segmentation, and efficient retrieval (Liu et al. 2026; Fang et al. 2025). Mem- α acquires memory construction policies through reinforcement learning and provides a competitive trained text-memory baseline (Wang et al. 2025). These methods primarily study how memory content is formed and recalled. MEMO extends this line to the read interface with a trained memory manager that selects evidence units and organizes their presentation for the final reader.

Structured and Lifecycle-Aware Memory

Another line of work models memory as a managed system with explicit structure and lifecycle operations. EverMemOS organizes episodic traces into higher-level memory objects with lifecycle control (Hu et al. 2026). MemoryOS separates short-term, medium-term, and long-term memory through migration rules (Kang et al. 2025). Graph and event memory systems such as MAGMA, SYNAPSE, ES-Mem, CAM, A-MEM, and Amory show that temporal, causal, entity, event, and narrative relations can support long-horizon reasoning (Jiang et al. 2026a,b; Zou et al. 2026; Li et al. 2025; Xu et al. 2025; Zhou et al. 2026). MEMO can consume structured records produced by these systems because evidence units preserve provenance, roles, priorities, and presentation requirements. It constructs working memory during the read stage so that structure in long-term storage can be used in a form suited to the current reader.

Visual and Optical Memory

Visual memory methods organize memory as two-dimensional artifacts. MemOCR converts textual memory into layout-aware visual pages and uses budget-aware learning to support long-horizon reasoning (Shi et al. 2026).

AgentOCR represents agent history as OCR-style images through optical self-compression (Feng et al. 2026). These studies show that images can carry memory content and that readers are sensitive to visual organization. MEMO places presentation-form selection inside unit-level management so the system can retain a textual precision anchor, use visual structure, and choose dual presentation when a unit needs both kinds of information.

Multi-Agent Memory and Context Routing

Multi-controller memory systems assign management operations to specialized modules. MIRIX coordinates multiple memory components for LLM agents (Wang and Chen 2025). G-Memory builds hierarchical graph memory for multi-agent systems (Zhang et al. 2025). AMA uses collaborative agents for adaptive memory (Huang et al. 2026). RCR-Router studies role-aware context selection under limited resources (Liu et al. 2025). These studies demonstrate the value of explicitly managing model input. MEMO trains a read policy at evidence-unit granularity and uses frozen-reader outcomes to evaluate the final text and visual memory artifact.

Method

Task Definition and Overview

Figure 2 illustrates the workflow of MEMO. Given a current query q , memory records $\mathcal{R}$, and a total memory budget B , MEMO constructs working memory $a = (T, V)$ for one reader call, where T is a text packet and V is an optional visual page. A trained evidence extractor first identifies source blocks relevant to the query and predicts character-offset highlights for key facts. A deterministic unit-materialization step copies the corresponding content from the original records, preserves its source links and highlights, and produces a set of evidence units $\mathcal{U}$. The memory manager then decides which units enter working memory, whether each unit uses text, image, or dual presentation, and how visual units are organized. All presentation decisions share one resource limit, $C_{\text{text}} + C_{\text{vis}} \leq B$.

Query-Conditioned Evidence Extraction

The existing evidence extraction methods based on memory passages or memory chunks are often too coarse-grained to directly support fine-grained memory management. For example, a document passage may contain only one sentence that is relevant to the query, while a long dialogue turn may include both key events and many irrelevant details. If the entire retrieved result is used as the memory context, such background content competes with truly useful facts for limited memory space. To address this issue, we use a query-conditioned evidence extractor that first locates useful content in the raw records, allowing subsequent steps to operate on smaller memory items with clearer semantics.

It outputs a set of source identifiers $\mathcal{S}$ for query-relevant memory chunks, together with character-level key spans $\mathcal{H}$ for each selected source. We denote this process as $(\mathcal{S}, \mathcal{H}) = E_\phi(q, \mathcal{R}, B)$. Each key span contains its start and end positions within the source chunk. The source record and these spans jointly determine the content of each evidence

unit. For each selected memory-chunk source, the system directly copies the corresponding content from the raw record and preserves the local context within a window of width w around the key span. The resulting evidence unit is the smallest memory item that MEMO can retain, reformat, or omit. It is defined as

$$u_i = (\text{id}, x_i, h_i, s_i, c_i). \quad (1)$$

The field x_i stores the evidence content, h_i stores the key span located by the extractor, and s_i records the source. The field c_i stores the priority rule for evidence retention. This rule depends on whether the content in x_i belongs to the key span located by h_i or to its associated context. It can require semantic information to be preserved in textual form, allow visual presentation to meet the budget constraint, or allow omission. As a result, subsequent modules can protect more critical semantic information when reducing memory.

Evidence Extractor Training

We construct extractor training examples from evidence annotations in the training sets of four benchmarks. We prioritize native evidence annotations from each benchmark and use deterministic weak labels when native span annotations are unavailable. In HotpotQA and 2WikiMultiHopQA, each context paragraph serves as a source chunk, and the official supporting facts (title, sentence id) determine the selected paragraphs and supporting sentences. For LoCoMo, each dialogue turn with a speaker and a timestamp serves as a source chunk. The official evidence turn identifiers in each question-answer entry determine the selected sources, and the training, validation, and test sets are split at the dialogue level. For ALFWorld, we read tasks and high-level actions from the official expert trajectories, and deterministically replay the actions to construct adjacent pre-execution and post-execution states. Each entity in the pre-execution state serves as a source chunk. Entities that change in the next step are selected by state differencing, and the full serialized state block is labeled as state evidence.

Each training target contains selected source identifiers, annotated character spans, and mappings from sources to evidence units. Let e^* denote the serialized extraction target. We train the extractor through full-parameter supervised finetuning to generate this structured output from the query, candidate records, and budget using

$$\mathcal{L}_{\text{ext}}(\phi) = - \sum_t \log p_\phi(e_t^* | e_{<t}^*, q, \mathcal{R}, B). \quad (2)$$

Joint Evidence Selection and Presentation

After evidence units are generated, the memory manager conditions on the current query, the evidence units, and the budget, and selects an action for each unit as its presentation form.

$$\mathcal{M} = \{\text{text}, \text{image}, \text{dual}, \text{drop}\}. \quad (3)$$

The action `drop` omits the unit from the current working memory. Omission saves resources, but it may also remove evidence needed for subsequent reasoning. For selected evidence units, the memory manager chooses different presentation forms according to their content. The action `text`

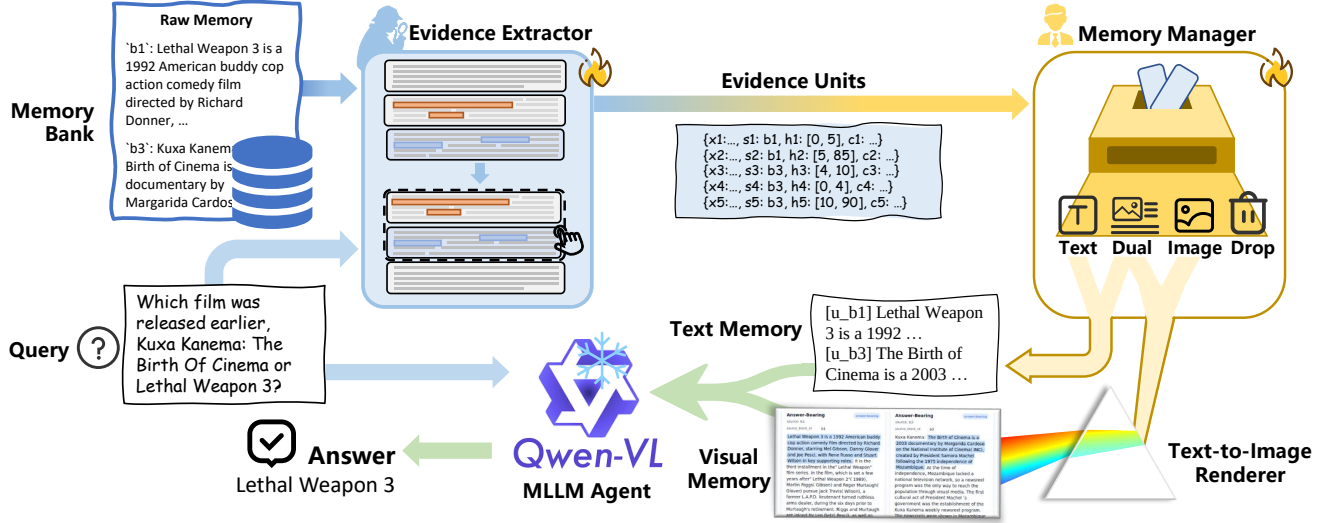


Figure 2: **Workflow of MEMO**. MEMO extracts evidence units from raw memory, plans their presentation as text, image, dual-channel, or dropped content, and builds a query-specific working memory for the multimodal LLM agent.

preserves a compact textual anchor in the text package T , image places the unit into the visual page V , and dual both preserves the textual anchor and adds the unit to the visual structure. Text preserves precise semantic information, while images preserve temporal, tabular, or grouping relations. However, images consume the visual budget and may reduce the recoverability of strings. The dual form retains a precise textual anchor and adds structural information, with its cost counted toward both textual and visual resources. For visual structures, the memory manager further plans template preferences, which control whether visual units are rendered as cards, timelines, tables, checklists, or column layouts, so that different memory views can be constructed according to the meaning of the evidence. For example, in multi-hop questions, answer-related spans can be preserved as text, while relations from multiple documents are organized with cards. In long dialogues, key entities can be preserved as text, while event order is presented with a timeline. In embodied trajectories, goals and action parameters can be highlighted, while state changes are organized as a checklist.

Because content selection, presentation form, and visual organization compete for the same budget, the memory manager plans them jointly with a single evidence-unit-level plan. The plan contains the proposed action and confidence for each unit, the target memory usage, and the preferred page templates. After joint planning, the memory manager outputs the textual anchors and visual-structure information for the current query that are needed to construct the working memory.

Memory Manager Training from Reader Outcomes

We construct training examples from HotpotQA, 2WikiMultiHopQA, LoCoMo, and ALFWorld. For the same query and evidence, we try keeping one evidence unit as text, placing it in an image, keeping both forms, or leaving it out of the current memory. All other units remain fixed, which allows us

to compare the effects of different counterfactual candidates. Each candidate memory is given to the same frozen reader to answer the question, and we record both the answer quality and the total memory usage. Candidates that violate the budget are removed. When multiple candidates yield similar answer quality, the one that uses fewer tokens is preferred. The final winning choice serves as the action label for that evidence unit.

Each action label is also associated with a confidence score. The score comes from the gap between the best action and the second-best action. Confidence is high when one presentation is clearly better and lower when two choices perform similarly. This value is scaled to a fixed range and only indicates how certain the action choice is for this training sample. Image and drop choices also pass a separate check based on actual reader results, preventing choices that may lose essential information from entering the training target.

The template label specifies how visual memory should be laid out. For each training sample, the template is determined by the answer quality of its counterfactual candidates. The training target stores an ordered list of candidate templates rather than a single template. During inference, the first template that satisfies the budget constraint is retained. The template preference is written into the same management plan together with the action and confidence.

The input of each training sample contains the query q , the evidence units $\mathcal{U}$, and the budget B , while the target y^* is the complete memory plan. The training objective of the memory manager is

$$\mathcal{L}_{\text{stf}}(\theta) = - \sum_t \log p_{\theta}(y_t^* | y_{<t}^*, q, \mathcal{U}, B). \quad (4)$$

Working Memory Building

Reading the memory plan. Working-memory construction takes the current query, the evidence units created by the

extractor, the total budget, and the memory manager’s plan as input. Each evidence unit contains content copied from the original memory together with its source and key passage. The plan gives a preferred presentation and confidence for every unit, as well as the target memory use and visual-template preferences. Under the shared budget, the builder decides which units enter the text packet, the visual page, both, or neither.

Preparing the text packet. Evidence units assigned text or dual become short text entries. The builder copies their key wording directly from the corresponding source records and keeps the unit and source identifiers with each entry, preserving the link to the original memory. The ordered entries form the text packet T . The unified text-token accounting rule gives its cost C_{text} , leaving the rest of the total budget for visual memory.

Preparing the visual page. Evidence units assigned image or dual become visual blocks containing a title, source, content, information type, and highlighted wording. The memory manager’s template preferences arrange these blocks as a timeline, table, checklist, cards, or columns. Once the text cost is known, the remaining budget determines the available page size, and the same page size determines the visual cost C_{vis} . The renderer then lays out the blocks as the visual page V , with $C_{\text{text}} + C_{\text{vis}} \leq B$.

Assembling the reader input. The text packet and visual page together form the final working memory $a = (T, V)$. The system joins the text entries into readable text labeled by unit identifier and renders the visual page as an image. It then gives the current query, text, and image to the frozen reader, which produces an answer from this query-specific working memory.

Experiments

Implementation Details

Both trained components, the evidence extractor and the memory manager, use Qwen2.5-1.5B-Instruct as their backbone. We conducted supervised fine-tuning with `verl` (Sheng et al. 2024). The reader used to train memory manager is InternVL3.5-4B-Instruct. All experiments were conducted on NVIDIA A100 GPUs.

Evaluation Settings

The HotpotQA and 2WikiMultiHopQA test sets come from official development sets with reference answers, each pre-split into non-overlapping validation and test sets. The LoCoMo test set contains two fully held-out conversations. ALFWorld test samples are next-action prediction instances derived from official valid_unseen expert trajectories. HotpotQA and 2WikiMultiHopQA evaluate multi-hop document reading, LoCoMo evaluates memory over long conversations, and ALFWorld evaluates whether task and state information retained in memory supports recovering the expert’s next action. This design gives all four benchmarks the same Reader output format and metrics, enabling a clear, consistent Overall score.

Baselines. We compare MEMO with three direct controls and five published baselines. No memory gives the Reader only the query. Text-only and Visual-only present the available memory directly as text or a rendered page, without a learned memory constructor. BM25 ranks textual evidence by lexical relevance to the query (Robertson and Zaragoza 2009). The learned text-memory baselines are Mem- α , which learns memory construction through reinforcement learning (Wang et al. 2025), and MemAgent, which reads the source in segments and repeatedly updates a compact text memory (Yu et al. 2025). The visual-memory baselines are AgentOCR, which compresses history into OCR-style images (Feng et al. 2026), and MemOCR, which constructs layout-aware visual memory (Shi et al. 2026). Each resulting memory is passed to the same frozen Reader so that the comparison reflects how the memory is constructed.

Readers. We evaluate every method with three frozen multimodal Readers: InternVL3.5-8B, Qwen3-VL-32B, and gpt-5.4-mini. They cover different model scales and families, allowing us to test whether a memory method depends on one particular backend. For a given Reader, all methods receive the same query and source memory, and the Reader remains unchanged throughout evaluation.

Memory budgets. We compare a strict 128-token budget with a setting that imposes no token budget. Under 128-token budget, every method must satisfy text tokens + visual tokens ≤ 128 , enabling a direct quality comparison under the same tight limit. In the no-budget setting, we impose no method-level memory limit and do not truncate the source memory. Execution still uses a 4,096-token LLM context window, but the complete source memory of every test example fits within it. No method is therefore forced to discard or compress content to meet an experimental cap, so this setting is effectively unrestricted.

Metrics. The result tables report normalized exact match (EM) and token-level F1 as percentages. After converting all answers to lowercase and removing punctuation and English articles, EM assigns a score only when the prediction exactly matches the reference answer, whereas F1 gives partial credit based on token overlap between the two answers. EM measures strict correctness, while F1 is more sensitive to partially correct content in multi-token answers. Using the same two metrics also allows a unified Overall score to be computed across the four tasks. For ALFWorld, we therefore evaluate whether the Reader can predict the expert’s next action from the task and the saved trajectory state, and compare the prediction with the reference action.

Main Results

Tables 1 and 2 compare MEMO with text-only, visual-only, and other memory methods under a tight 128-token budget and with no token budget.

128-token limit. Under the strict 128-token limit, MEMO achieves the best Overall EM and F1 with all three readers. With InternVL3.5-8B, it improves over the strongest baseline by 7.30 EM points and 7.22 F1 points. The gains remain

Reader	Memory Type	Method	Overall		2Wiki		HotpotQA		LoCoMo		ALFWorld	
			EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
InternVL3.5 8B	None	No memory	13.51	17.60	20.99	23.86	8.56	15.81	0.32	4.26	0.00	1.92
	Text	Text-only	32.93	45.36	36.66	47.84	43.53	58.59	16.56	43.37	1.37	10.34
		BM25	31.60	43.93	34.91	45.77	41.91	56.83	16.88	43.75	1.75	11.44
		Mem- α	30.24	43.24	32.98	45.25	41.02	56.35	<u>18.40</u>	<u>44.88</u>	0.95	9.43
		MemAgent	37.33	51.95	41.29	<u>55.98</u>	50.97	68.35	13.53	39.31	0.00	6.87
	Visual	Visual-only	29.67	42.45	32.63	43.95	40.26	57.47	17.21	42.90	0.05	6.72
		AgentOCR	<u>40.53</u>	<u>52.60</u>	<u>44.70</u>	55.27	<u>54.58</u>	71.02	22.40	49.12	0.71	6.63
		MemOCR	16.08	22.08	24.07	28.24	11.45	20.88	1.95	10.51	0.33	5.24
	Managed	MEMO (Ours)	47.83	59.82	58.55	68.88	54.93	<u>70.09</u>	17.21	43.05	<u>1.68</u>	<u>10.62</u>
Qwen3-VL 32B	None	No memory	15.46	20.40	19.06	22.98	18.18	26.22	0.97	3.11	0.00	2.64
	Text	Text-only	40.23	52.99	45.59	56.26	51.16	66.90	26.41	51.20	1.90	13.82
		BM25	38.32	51.07	43.09	53.66	49.41	65.19	25.54	<u>50.50</u>	1.53	13.59
		Mem- α	41.35	52.31	47.09	56.05	53.16	67.96	24.46	48.99	0.46	8.23
		MemAgent	<u>47.88</u>	<u>62.00</u>	<u>56.42</u>	<u>69.80</u>	58.16	75.38	23.92	48.71	1.61	10.20
	Visual	Visual-only	22.18	32.60	26.53	35.89	26.83	42.20	8.87	26.43	0.00	2.82
		AgentOCR	32.88	45.54	38.57	48.87	41.02	60.39	13.64	37.93	0.00	5.21
		MemOCR	20.61	28.06	27.04	32.08	21.37	33.49	2.27	12.14	0.05	5.90
	Managed	MEMO (Ours)	50.23	63.84	62.20	73.91	<u>55.83</u>	<u>72.52</u>	<u>25.76</u>	50.22	<u>1.74</u>	<u>13.81</u>
gpt-5.4-mini	None	No memory	20.40	22.98	27.17	28.51	20.63	26.67	0.32	0.45	0.00	0.27
	Text	Text-only	47.49	57.24	55.58	62.42	58.23	71.71	30.30	<u>54.99</u>	0.71	10.45
		BM25	44.52	54.17	51.16	57.75	56.13	69.48	30.19	54.98	0.49	10.67
		Mem- α	46.34	55.65	54.12	60.95	57.29	70.58	29.98	53.41	0.07	7.45
		MemAgent	<u>55.31</u>	<u>65.81</u>	<u>66.88</u>	<u>75.28</u>	65.20	80.16	26.52	51.40	0.20	6.49
	Visual	Visual-only	36.24	46.54	42.12	49.96	44.31	60.01	27.27	49.53	1.11	6.93
		AgentOCR	41.74	52.75	47.38	55.75	53.28	68.80	30.95	54.31	0.73	9.63
		MemOCR	28.96	37.53	35.11	41.42	33.69	46.20	10.50	26.67	<u>1.24</u>	8.37
	Managed	MEMO (Ours)	57.78	68.62	71.68	80.06	<u>63.88</u>	<u>78.14</u>	<u>30.41</u>	55.04	2.12	12.18

Table 1: Results under 128-token budget limitation, reporting normalized exact match (EM) and token F1.

clear with the larger readers: MEMO leads the strongest baseline by 2.35 EM and 1.84 F1 points with Qwen3-VL-32B, and by 2.47 EM and 2.81 F1 points with gpt-5.4-mini. The improvement is therefore not tied to one particular reader. MEMO is especially strong on 2Wiki, where it ranks first in both metrics for every reader, and it remains first or second on HotpotQA. Although another baseline is better on some individual LoCoMo and ALFWorld scores, MEMO keeps the clearest advantage when all test examples are considered together.

No token budget. When no token budget is imposed, MEMO uses only 83.93 tokens on average for every reader. This is 50.5–50.7% fewer tokens than MemAgent, the next smallest nonzero-memory baseline, and 66.9% fewer than Text-only. Even with this lower memory use, MEMO remains near the top in overall answer quality. It obtains the best Overall F1 with InternVL3.5-8B and ranks second on its Overall EM by only 0.06 points. With Qwen3-VL-32B and gpt-5.4-mini, it is second on both Overall metrics, and its largest gap from the best result is only 0.43 points. Thus, MEMO is first or second on every Overall EM and F1 result while using the least memory among all methods that

provide memory.

Overall takeaway. The two settings show complementary advantages. Under a hard limit, MEMO gets more useful information into the same small budget and produces better answers. When much more context is available, it does not spend tokens simply because it can. It preserves near-best answer quality with substantially less memory. The same pattern across three different readers suggests that the advantage comes from MEMO’s memory management rather than from a favorable match to one reader.

Component Ablation

Table 3 studies the two trainable components under the same 128-token budget with gpt-5.4-mini.

With both components disabled, the system reaches 47.49 EM and 57.24 F1 using 126.22 memory tokens. Enabling only the evidence extractor raises EM by 6.07 points and F1 by 8.85 points. Meanwhile, memory use falls to 70.12 tokens, a 44.4% reduction. This shows that selecting query-relevant evidence can improve the reader’s answer while removing substantial unnecessary memory.

Reader	Memory Type	Method	Overall			2Wiki		HotpotQA		LoCoMo		ALFWorld	
			EM	F1	Tokens ↓	EM	F1	EM	F1	EM	F1	EM	F1
InternVL3.5 8B	None	No memory	13.42	17.47	0.00	20.91	23.77	8.40	15.60	0.32	4.17	0.00	1.80
	Text	Text-only	48.99	60.82	253.89	57.52	67.49	<u>60.60</u>	<u>75.73</u>	18.18	44.90	1.21	10.29
		BM25	49.84	<u>61.72</u>	253.89	58.57	<u>68.58</u>	61.36	76.28	17.86	44.80	1.75	11.41
		Mem- α	49.61	60.88	267.24	<u>58.92</u>	68.07	60.33	75.42	<u>20.45</u>	<u>47.20</u>	0.71	8.94
		MemAgent	45.17	58.88	<u>170.23</u>	54.52	67.73	54.33	71.42	11.69	38.03	0.00	6.47
	Visual	Visual-only	36.65	50.75	4096.00	39.97	53.08	50.18	68.23	19.16	46.17	0.77	8.04
		AgentOCR	42.29	53.94	177.02	47.06	56.88	56.33	72.87	22.40	47.36	0.71	6.52
		MemOCR	45.86	57.59	705.66	52.58	62.57	58.98	74.67	19.48	46.00	0.55	7.72
	Managed	MEMO (Ours)	<u>49.78</u>	62.26	83.93	60.48	71.27	58.02	73.69	18.49	45.78	<u>1.53</u>	<u>10.75</u>
	Qwen3-VL 32B	None	No memory	16.15	20.98	0.00	20.17	23.80	18.55	26.71	0.97	3.27	0.00
Text		Text-only	53.13	66.76	253.89	63.92	75.55	62.22	<u>78.74</u>	26.95	<u>53.16</u>	<u>1.92</u>	14.45
		BM25	53.29	67.14	253.89	64.15	76.06	62.52	79.13	26.95	52.79	1.59	<u>14.46</u>
		Mem- α	52.33	64.96	267.35	62.90	74.26	<u>62.25</u>	78.37	24.68	49.84	0.44	8.28
		MemAgent	52.90	66.40	<u>169.55</u>	<u>65.12</u>	77.08	60.14	77.48	22.40	48.39	1.21	10.15
Visual		Visual-only	41.86	59.12	4096.00	50.60	67.77	49.64	70.59	17.86	44.57	0.00	8.48
		AgentOCR	43.01	56.02	177.02	51.43	62.00	51.55	70.57	22.40	48.14	0.16	7.20
		MemOCR	51.30	64.20	705.66	62.61	72.83	59.84	78.25	20.13	46.05	0.27	9.07
Managed		MEMO (Ours)	<u>53.27</u>	<u>66.77</u>	83.93	65.92	<u>77.06</u>	59.30	76.11	<u>26.62</u>	53.27	1.97	14.61
gpt-5.4-mini		None	No memory	20.45	23.05	0.00	27.18	28.70	20.77	26.57	0.32	0.44	0.00
	Text	Text-only	58.43	69.29	253.89	71.34	79.41	67.16	82.05	31.49	<u>57.05</u>	0.77	10.57
		BM25	58.94	69.74	253.89	<u>72.28</u>	80.17	67.43	82.39	30.52	55.67	0.55	10.50
		Mem- α	57.56	67.98	267.97	70.07	78.19	66.86	81.59	30.52	54.86	0.16	7.43
		MemAgent	57.34	67.91	<u>169.65</u>	70.12	78.52	66.30	81.67	26.62	51.99	0.33	6.15
	Visual	Visual-only	58.62	70.27	4096.00	71.72	<u>80.88</u>	<u>67.51</u>	83.04	29.87	56.06	0.27	10.17
		AgentOCR	48.48	60.02	177.02	55.69	64.22	60.79	76.51	32.79	55.32	1.32	12.90
		MemOCR	57.76	68.71	705.66	69.58	78.48	67.65	<u>82.52</u>	29.55	53.63	<u>1.75</u>	9.61
	Managed	MEMO (Ours)	<u>58.86</u>	<u>69.84</u>	83.93	73.06	81.48	65.02	79.52	<u>31.80</u>	57.58	1.99	<u>12.20</u>

Table 2: Results without source-memory truncation under the 4,096-token execution window, reporting normalized exact match (EM) and token F1.

Evidence Extractor	Memory Manager	Overall		
		EM	F1	Tokens
✗	✗	47.49	57.24	126.22
✓	✗	53.56	66.09	70.12
✓	✓	57.78	68.62	81.07

Table 3: Component ablation under the 128-token budget with gpt-5.4-mini. Tokens denotes average memory use.

Adding the memory manager on top of the evidence extractor further raises EM from 53.56 to 57.78 and F1 from 66.09 to 68.62, gains of 4.22 and 2.53 points. Memory use rises moderately from 70.12 to 81.07 tokens because the memory manager keeps a slightly richer working memory, but remains well below the 126.22 tokens used when both components are disabled. This shows that the memory manager does not simply minimize token count: it spends a few additional tokens when they improve answers.

Compared with disabling both components, the complete system improves EM by 10.29 points and F1 by 11.38 points

while using 35.8% fewer memory tokens. The extractor provides the first large improvement, and the memory manager adds a further gain, so the two modules make distinct, complementary contributions to the final result.

Conclusion

We propose MEMO, a query-conditioned memory read-out method that converts memory into evidence units and jointly selects content, textual or visual presentation, and layout under a token budget. Across four benchmarks, MEMO achieves the best overall EM and F1 under the 128-token constraint when using all three readers. Under the 4,096-token window, it uses 83.93 tokens on average while ranking first or second on every overall metric. Ablation studies show that evidence extraction provides the main gains in compression and accuracy by preserving query-relevant information before presentation choices, while the memory manager further improves quality. These results indicate that the proposed evidence-level multimodal organization serves as an effective and efficient memory interface.

References

- Fang, J.; Deng, X.; Xu, H.; Jiang, Z.; Tang, Y.; Xu, Z.; Deng, S.; Yao, Y.; Wang, M.; Qiao, S.; Chen, H.; and Zhang, N. 2025. LightMem: Lightweight and Efficient Memory-Augmented Generation.
- Feng, L.; Yang, F.; Chen, F.; Cheng, X.; Xu, H.; Wan, Z.; Yan, M.; and An, B. 2026. AgentOCR: Reimagining Agent History via Optical Self-Compression. *arXiv:2601.04786*.
- Ho, X.; Duong Nguyen, A.-K.; Sugawara, S.; and Aizawa, A. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In Scott, D.; Bel, N.; and Zong, C., eds., *Proceedings of the 28th International Conference on Computational Linguistics*, 6609–6625. Barcelona, Spain (Online): International Committee on Computational Linguistics.
- Hu, C.; Gao, X.; Zhou, Z.; Xu, D.; Bai, Y.; Li, X.; Zhang, H.; Li, T.; Zhang, C.; Bing, L.; and Deng, Y. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. *arXiv:2601.02163*.
- Huang, W.; Wang, Z.; Lin, H.; Wang, S.; Xu, B.; Li, Q.; Zhu, B.; Yang, L.; and Qin, C. 2026. AMA: Adaptive Memory via Multi-Agent Collaboration. *arXiv:2601.20352*.
- Jiang, D.; Li, Y.; Li, G.; and Li, B. 2026a. MAGMA: A Multi-Graph Based Agentic Memory Architecture for AI Agents. *arXiv:2601.03236*.
- Jiang, H.; Chen, J.; Pan, Y.; Chen, L.; You, W.; Zhou, Y.; Zhang, R.; Abate, Y.; and Liu, T. 2026b. SYNAPSE: Empowering LLM Agents with Episodic-Semantic Memory via Spreading Activation. *arXiv:2601.02744*.
- Kang, J.; Ji, M.; Zhao, Z.; and Bai, T. 2025. Memory OS of AI Agent. *arXiv:2506.06326*.
- Li, R.; Zhang, Z.; Bo, X.; Tian, Z.; Chen, X.; Dai, Q.; Dong, Z.; and Tang, R. 2025. CAM: A Constructivist View of Agentic Memory for LLM-Based Reading Comprehension. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Liu, J.; Kong, Z.; Yang, C.; Yang, F.; Li, T.; Dong, P.; Nanjey, J.; Tang, H.; Yuan, G.; Niu, W.; Zhang, W.; Zhao, P.; Lin, X.; Huang, D.; and Wang, Y. 2025. RCR-Router: Efficient Role-Aware Context Routing for Multi-Agent LLM Systems with Structured Memory. *arXiv:2508.04903*.
- Liu, J.; Su, Y.; Xia, P.; Han, S.; Zheng, Z.; Xie, C.; Ding, M.; and Yao, H. 2026. SimpleMem: Efficient Lifelong Memory for LLM Agents. *arXiv:2601.02553*.
- Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In Ku, L.-W.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13851–13870. Bangkok, Thailand: Association for Computational Linguistics.
- Robertson, S.; and Zaragoza, H. 2009. *The Probabilistic Relevance Framework*. Hanover, MA, USA: Now Publishers Inc. ISBN 978-1-60198-308-4.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv: 2409.19256*.
- Shi, Y.; Liu, S.; Yang, Y.; Mao, W.; Chen, Y.; GU, Q.; Su, H.; Cai, X.; Wang, X.; and Zhang, A. 2026. MemOCR: Layout-Aware Visual Memory for Efficient Long-Horizon Reasoning. *arXiv:2601.21468*.
- Shridhar, M.; Yuan, X.; Cote, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2020. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*.
- Wang, Y.; and Chen, X. 2025. MIRIX: Multi-Agent Memory System for LLM-Based Agents. *arXiv:2507.07957*.
- Wang, Y.; Takanobu, R.; Liang, Z.; Mao, Y.; Hu, Y.; McAuley, J.; and Wu, X. 2025. Mem- α : Learning Memory Construction via Reinforcement Learning. *arXiv:2509.25911*.
- Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv:2502.12110*.
- Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In Riloff, E.; Chiang, D.; Hockenmaier, J.; and Tsujii, J., eds., *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2369–2380. Brussels, Belgium: Association for Computational Linguistics.
- Yu, H.; Chen, T.; Feng, J.; Chen, J.; Dai, W.; Yu, Q.; Zhang, Y.-Q.; Ma, W.-Y.; Liu, J.; Wang, M.; and Zhou, H. 2025. MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent. In *The Fourteenth International Conference on Learning Representations*.
- Yu, Y.; Yao, L.; Xie, Y.; Tan, Q.; Feng, J.; Li, Y.; and Wu, L. 2026. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. *arXiv:2601.01885*.
- Zhang, G.; Fu, M.; Wang, K.; Wan, G.; Yu, M.; and Yan, S. 2025. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Zhang, S.; Wang, J.; Zhou, R.; Liao, J.; Feng, Y.; Zhang, W.; Wen, Y.; Li, Z.; Xiong, F.; Qi, Y.; Tang, B.; and Wen, M. 2026. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. *arXiv:2601.03192*.
- Zhou, Y.; Guo, X.; Bayar, B.; and Sengamedu, S. H. 2026. Amory: Building Coherent Narrative-Driven Agent Memory through Agentic Reasoning. *arXiv:2601.06282*.
- Zou, H.; Sun, T.; He, C.; Tian, Y.; Li, Z.; Jin, L.; Liu, N.; Zhong, J.; and Wei, K. 2026. ES-Mem: Event Segmentation-Based Memory for Long-Term Dialogue Agents. *arXiv:2601.07582*.